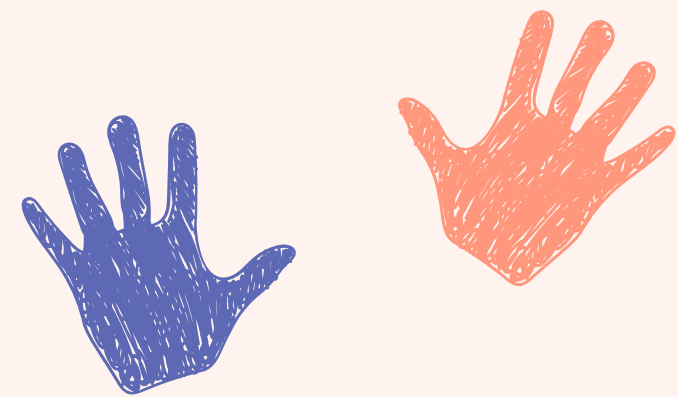
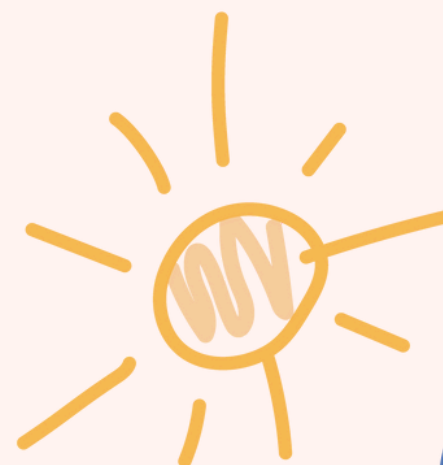
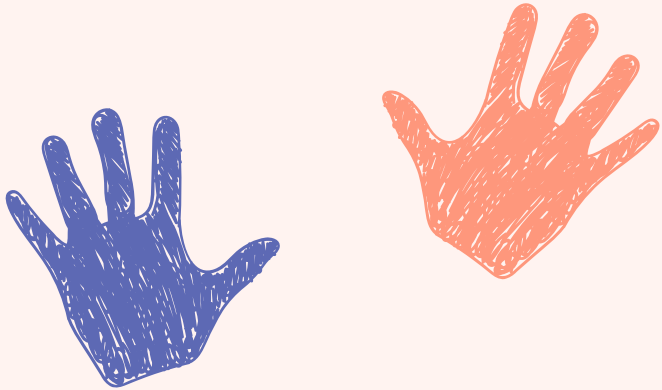




Memory Card Game

A Fun & Educational Matching Game for Kids





Presented by:

press F

Team Members:

Arda Köylü, Esmanur Sevim, Rutvik Kachhad,
Manan Khunt, Nikunj Vaghasiya



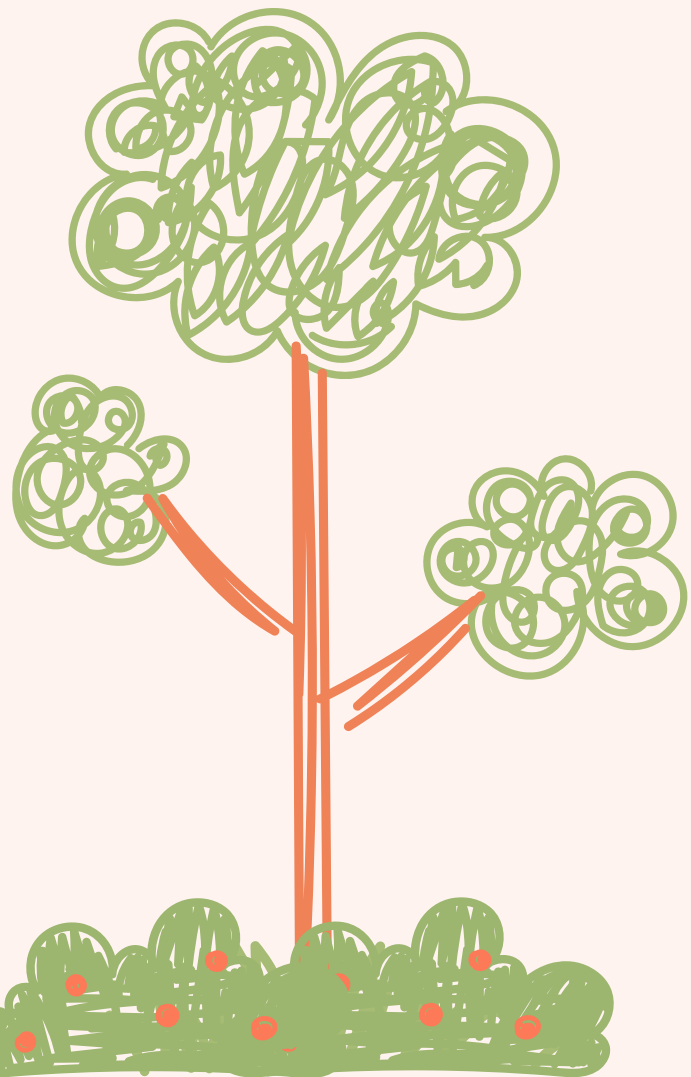
Memory Card Game

Our memory card game is a simple, fun, and safe matching game for children aged 3 to 7.

Kids flip animal cards, find pairs, and boost memory and focus skills while playing!

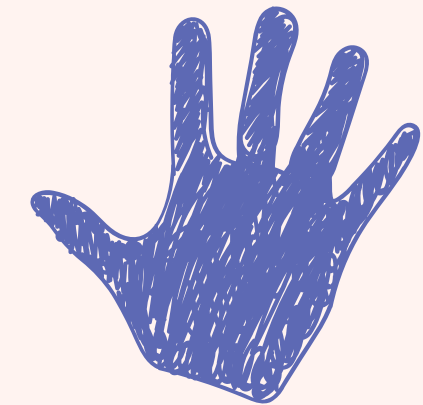
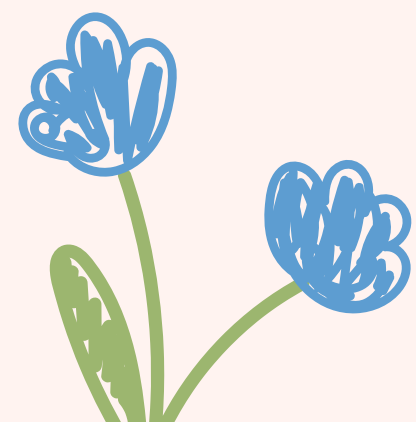
What is our goal?

To help children develop their memory, attention, visual recognition, and matching skills, all while enjoying a visually appealing experience





well
done!



Who is it For?

Children

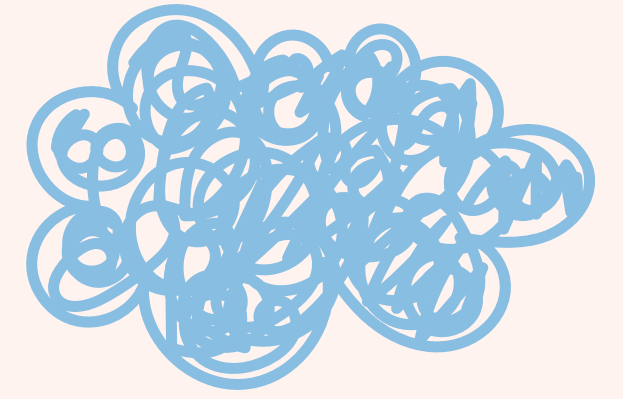
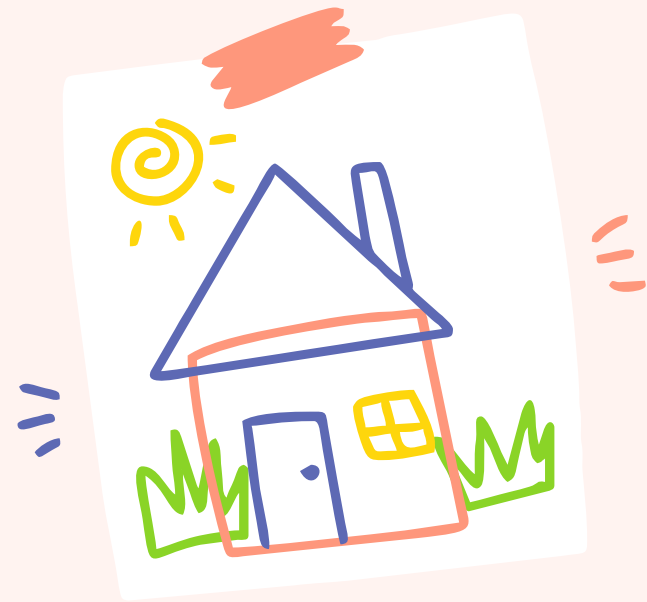
Designed for young children aged 3–7

Kindergarten

Perfect for kindergarten and early primary kids

Parents

Parents looking for safe, ad-free, educational fun



Why This Version



Users get feedback from
their inputs which
enhances the gameplay



Including difficulty selection
and a point system to provide
the users a motive to better
themselves



Gameplay





Development Tools

Godot Engine



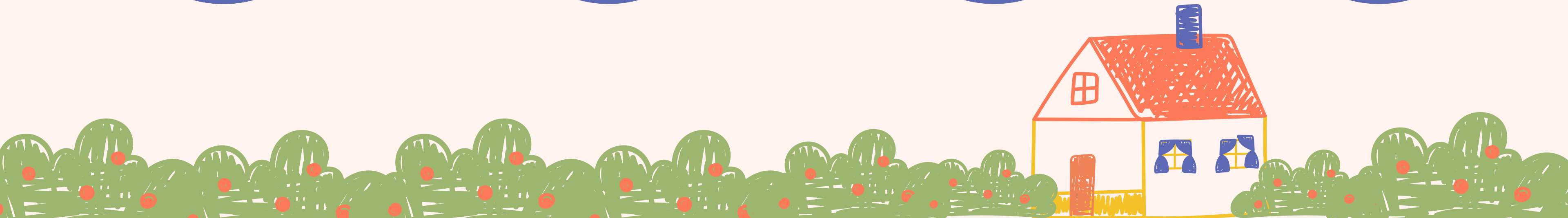
Android Studio

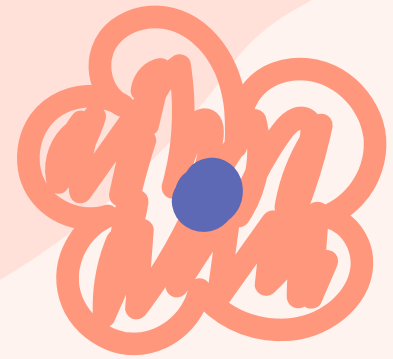


Gitlab



Canva





Agile Methods We Have Used



We organized our work in weekly sprints to set short-term goals and track our progress regularly.



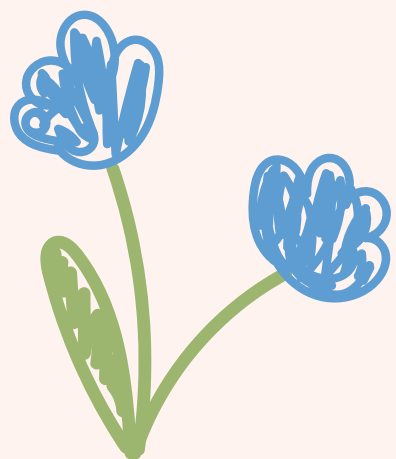
We used pair programming to solve problems collaboratively and improve code quality through constant feedback.



We managed our codebase using GitLab, allowing us to work in parallel, track changes, and handle merges efficiently.



We broke down complex tasks into smaller, manageable units to distribute responsibilities clearly and maintain steady progress.





Challenges

Settling Card Mechanics

Settling Point System

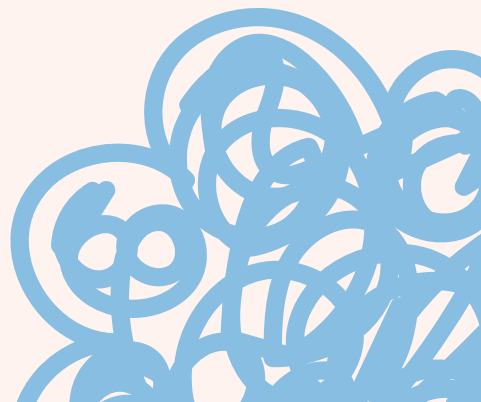
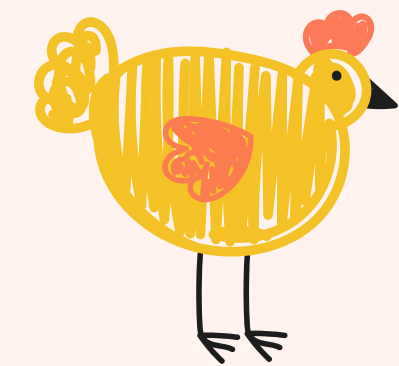
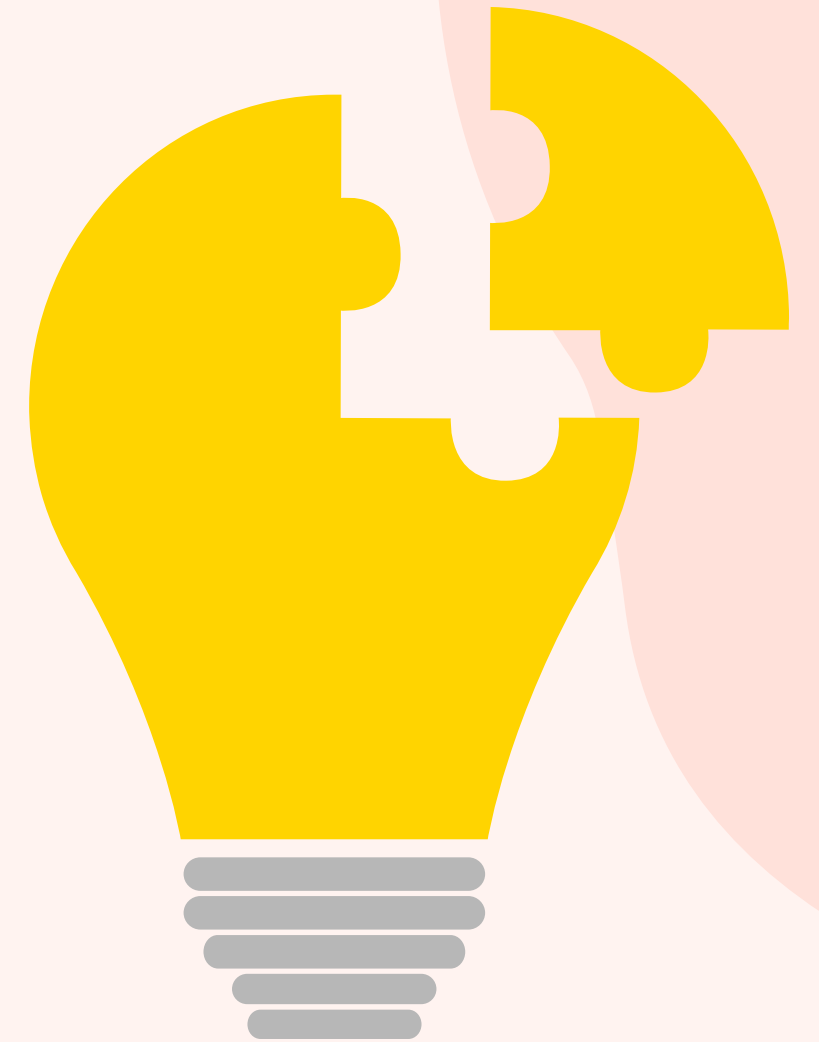
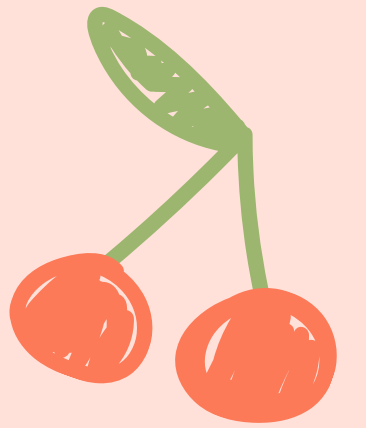
Adjusting array usage

Solutions

Starting from a single card then
adding multiple pairs

Adding print statements and
updating points on each pair

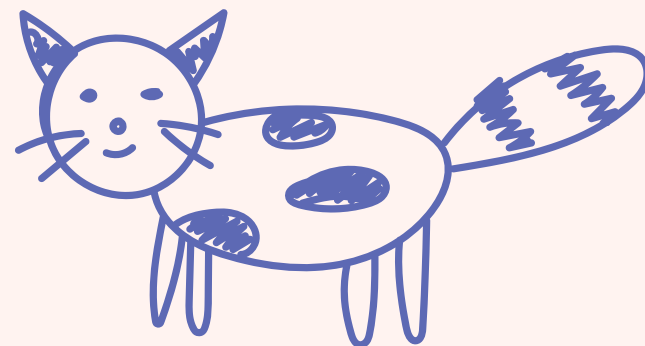
Instead of deleting the paired cards,
make them invisible and disabling
interactions



Task Division

Game Development

- Initialization of the Game: Arda, Esma
- Prototype Pitch: Arda, Esma
- Design of the Cards: Esma
- Pointing System: Arda
- Creating the First Level: Arda, Nikunj
- Main Screen & Screen Transitions: Arda, Esma
- HUD-timer and clicks, value transportation: Arda, Manan, Nikunj
- Android APK / PC Executable: Arda
- UI Design(buttons, screen design, color palette): Arda, Esma
- Audio Integration(background music, flip sound): Arda, Nikunj

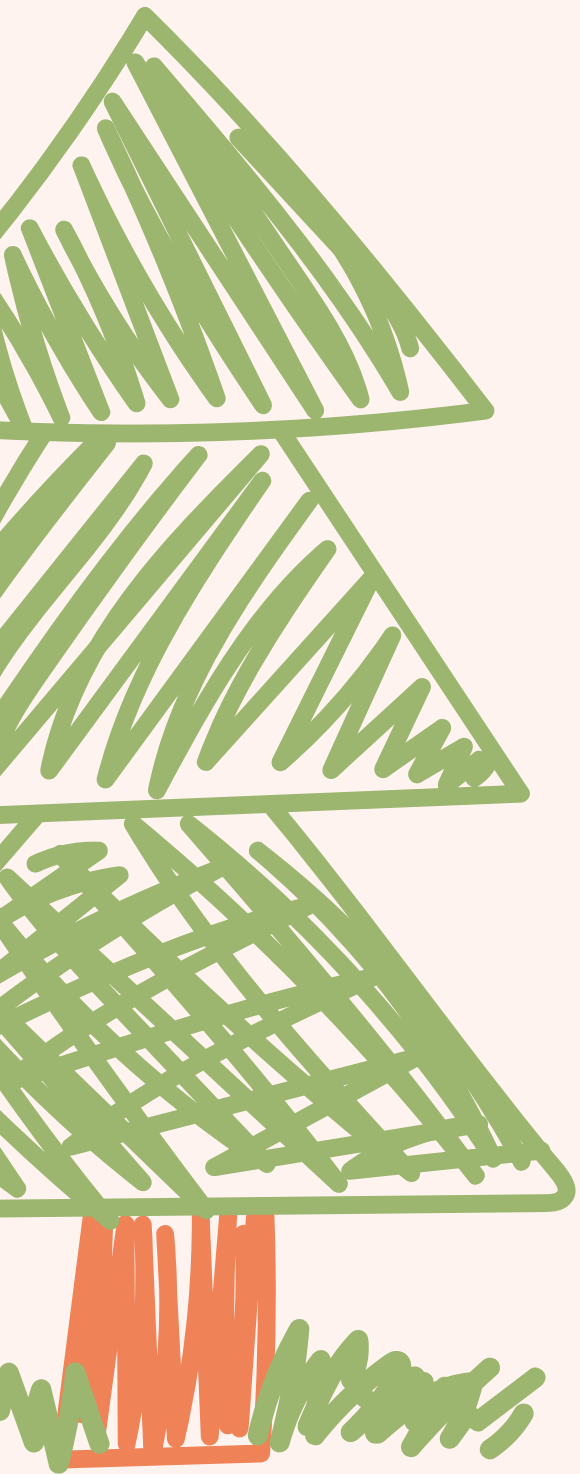


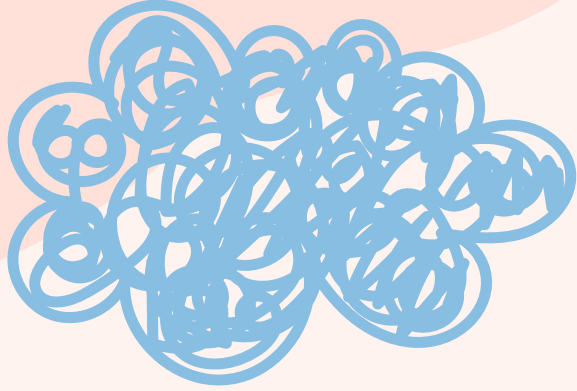
Documentation

- Requirements Documentation: Rutvik, Esma, Arda
- Architecture Documentation: Esma, Arda
- Test Documentation: Esma, Arda
- Acceptance Documentation: Esma, Arda
- User Documentation: Esma, Arda
- Project Documentation: Manan, Rutvik
- Mapping/Declaration of Individual Performance: Esma, Arda
- Polishing the Documentation before Test and Final Submission: Esma, Arda
- Checking if the documentation content meets the standards in the lecture content: Manan, Rutvik

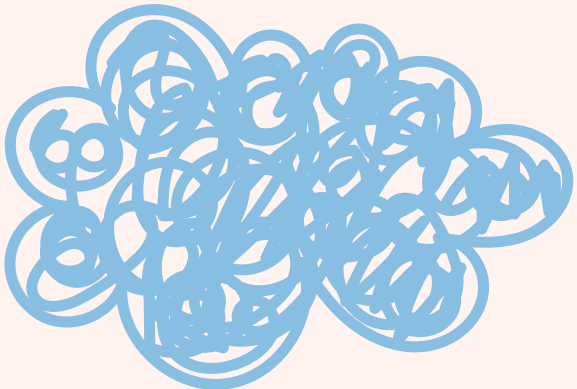
Final Presentation

- Preparing the content, materials (slideshow, screen recordings) : Esma, Arda





Technical Highlights



Game Logic

Includes a main menu and options to quit the game and select levels. In each level there are different amount of cards displayed. Player is able to see their click count and their time left at the level. After the level is finished a level completion popup is displayed.

Implementation Choices

After settling the individual card mechanics, the cards are displayed in an array on the levels. The total score is only displayed at the end of the level, even though its updated after each pairing.

Script Management

The game logic is mainly handled in GameManager script the point system is created by the fetched values from the other scripts.

Card script is handling independent card mechanics.

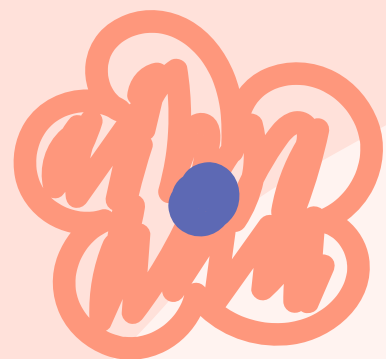
Game script is using the functionalities from the other scripts and has predefined values that gets overridden by the subscenes of Game scene.

Hud script handles the timers, and displayed elements.

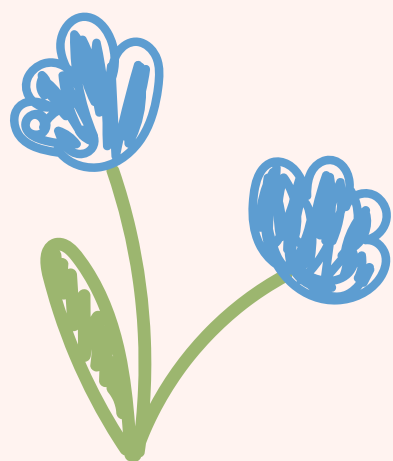
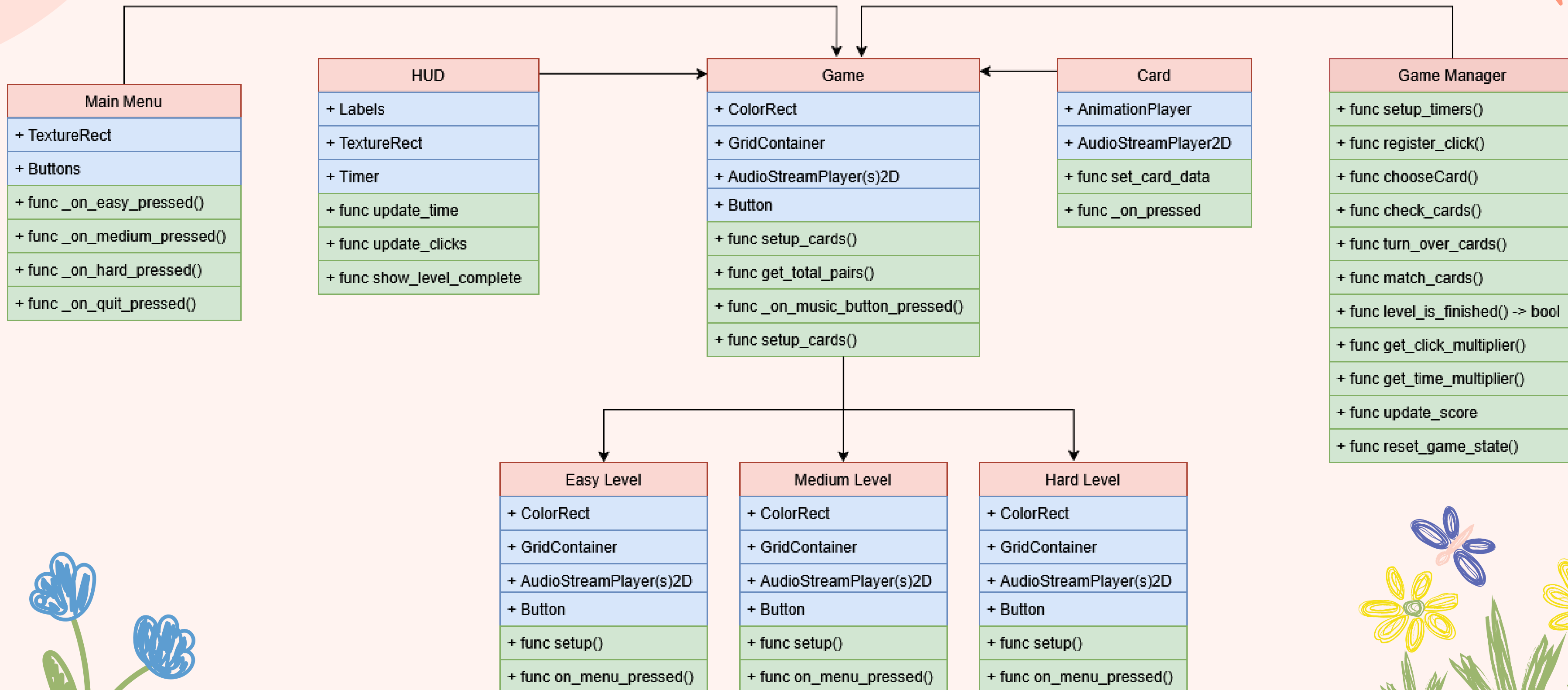
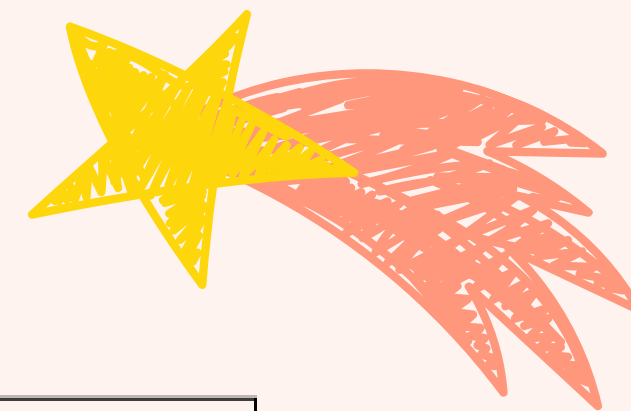
Design Choices

A soft color palette was selected to create a friendly and inviting atmosphere for children. Rounded UI elements (e.g., buttons, cards) were used to enhance a playful and approachable look. Integrated audio cues for immediate feedback on player actions, improving responsiveness and engagement.





UML Diagram



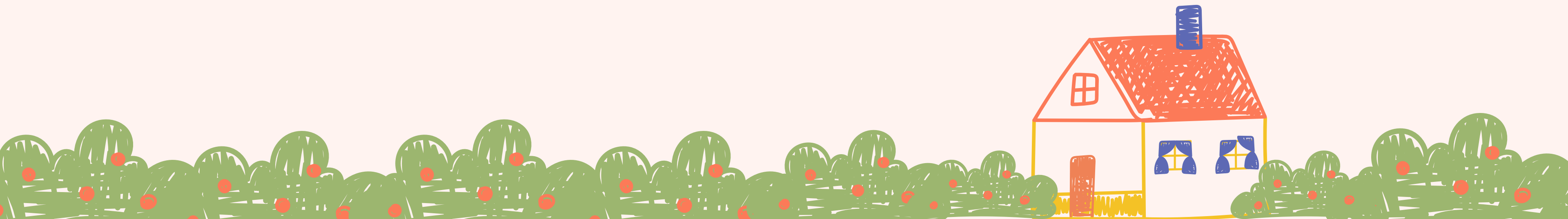


Code Highlights



```
▼ func _on_hard_pressed():  
    >| GameManager.register_click()  
    >| GameManager.reset_game_state()  
    >| get_tree().change_scene_to_file("res://levelHard.tscn")  
  
▼ func _on_quit_pressed():  
    >| GameManager.register_click()  
    >| get_tree().quit()
```

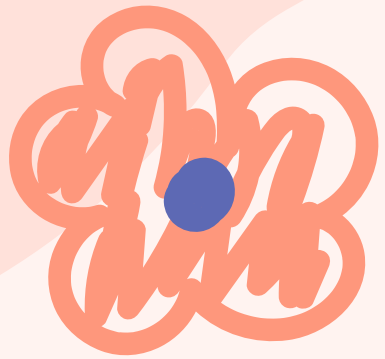
main_menu.gd



Code Highlights

```
▼ func setup_cards():  
    >| cards.clear()  
    >| grid.columns = cards_per_row  
▼ >| for suit in range(1, total_suits + 1):  
▼ >| >| for value in range(1, total_pairs + 1):  
    >| >| >| var card = CardScene.instantiate()  
    >| >| >| card.set_card_data(suit, value)  
    >| >| >| card.set_game(self)  
    >| >| >| cards.append(card)  
    >| cards.shuffle()  
▼ >| for card in cards:  
    >| >| grid.add_child(card)
```

game.gd

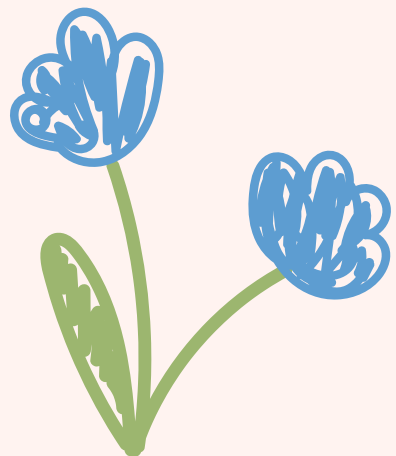


Code Highlights



```
▼ func set_card_data(s: int, v: int):  
    >| suit = s  
    >| value = v  
    >| face = load("res://assets/Assets 140x190 px/animal-%d-%d.png" % [suit, value])  
    >| back = GameManager.CardBack  
    >| replacement = GameManager.MatchedCardReplacementTexture  
    >| texture_normal = back
```

card.gd



Code Highlights

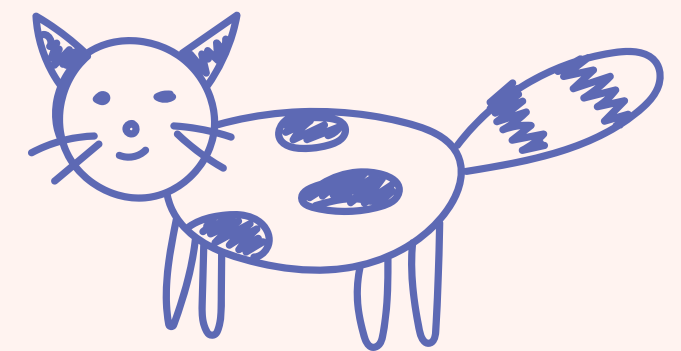
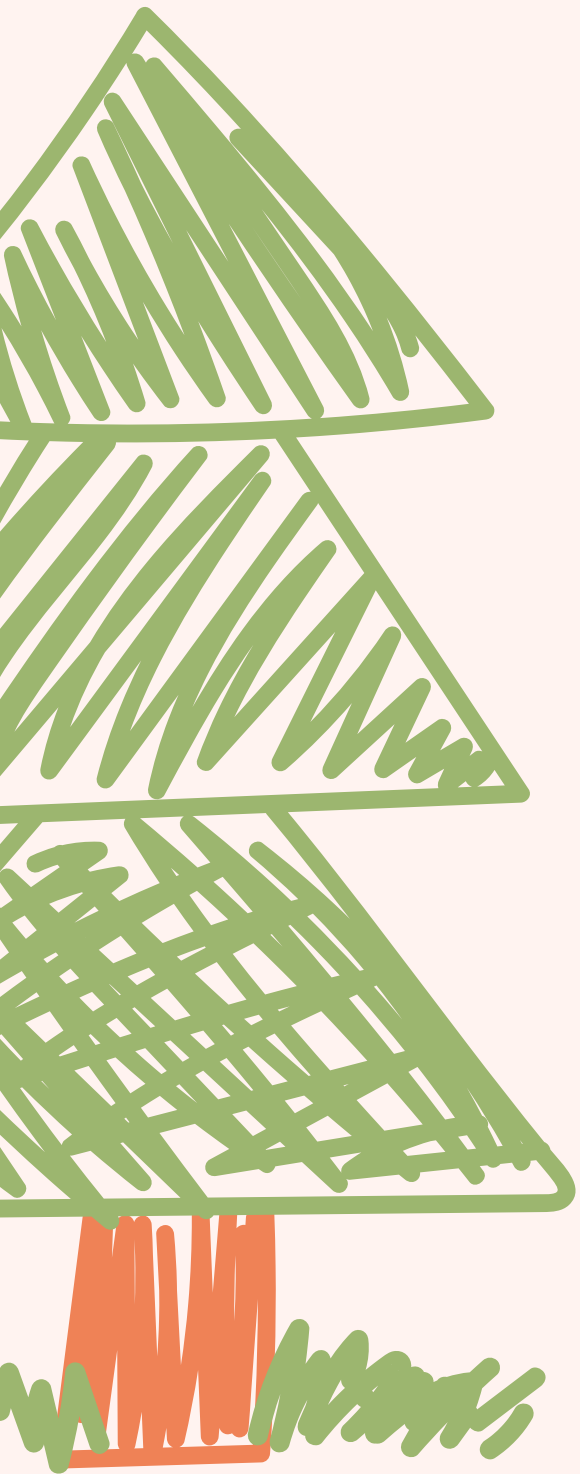
```
▼ func match_cards() -> void:
▼ >|   if has_node("/root/Game/MatchSound"):
>|   >|   get_node("/root/Game/MatchSound").play()

>|   card1.texture_normal = MatchedCardReplacementTexture
>|   card2.texture_normal = MatchedCardReplacementTexture

>|   card1 = null
>|   card2 = null

>|   found_pair_count += 1
>|   update_score()
>|
>|   var hud = get_node("/root/Game/HUD")
>|   var game_scene = get_tree().current_scene
▼ >|   if game_scene is Game:
▼ >|   >|   if level_is_finished():
>|   >|   >|   print("Found all pairs!")
>|   >|   >|   hud.show_level_complete_popup(score)
```

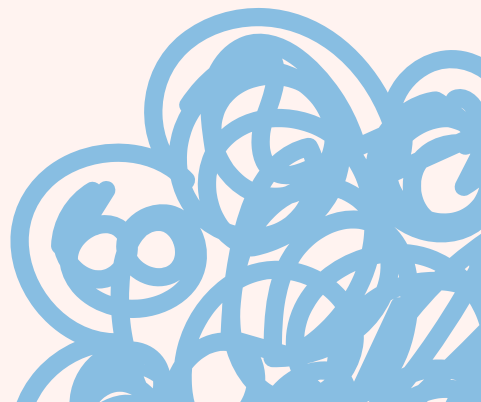
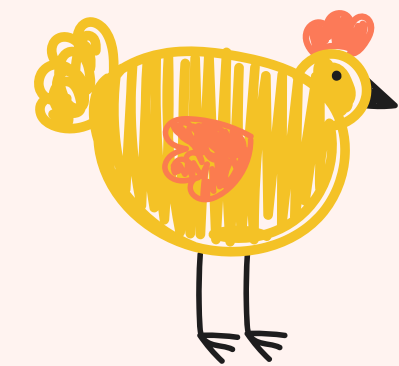
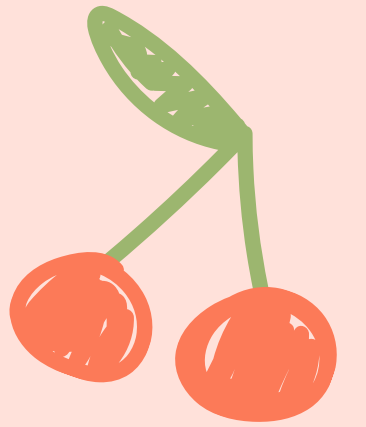
GameManager.gd



Code Highlights

```
▼ func level_is_finished() -> bool:  
    >| var game_scene = get_tree().current_scene  
▼ >| if game_scene is Game:  
    >| >| return found_pair_count == game_scene.total_pairs  
    >| return false
```

GameManager.gd



Reflection



What Went Well

- We had smooth collaboration through version control and regular communication.
- Pair programming helped us solve problems quickly and learn from each other.
- The core gameplay was functional early on, which gave us time to polish visuals and logic.
- UI design and scaling worked well across screen sizes, including mobile testing.

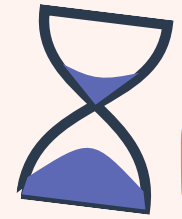


What Didn't Go So Well

- Initial task delegation lacked clear structure, which led to delays during the early stages of the project.
- Varying levels of engagement among team members resulted in an uneven workload and impacted overall progress.
- We underestimated the time needed for preparing the final presentation and reviewing the documentation thoroughly.

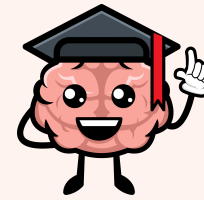


Reflection



Further Development

- Add more levels with increasing difficulty and different card themes.
- Introduce a leaderboard or score history system.
- Implement a multiplayer or versus mode for competitive play.
- Add language options to make the game more accessible.



What We Learned

- How to use GDScript and organize scenes in the Godot Engine effectively.
- The importance of clear task planning and communication in teamwork.
- How to test and scale UI for different devices, including mobile.
- How to write structured documentation and reflect individual contributions clearly.





Thank
You

